

Smart Faculty Access and Student Assistant System

INVESTIGATOR NAME: **P.Shakiththiyan**

STUDENT REGISTRATION NUMBER: **CS /2020/063**

CSCI 43018 - Research Project (2023/2024)

Project Proposal

B.Sc. (Hons) in Computer Science

SUPERVISOR: **Mr. Kesavan Selvarajah**

CONTENTS

1.RESEARCH PROBLEM	3
2.BACKGROUND	3
3.AIMS & OBJECTIVES	4
4.METHODOLOGY	5
1. System Implementation	5
2. AI & NLP TOOLS	6
5.TIME FRAME	7
6.ETHICS & PRIVACY	9
7.REFERENCES	10

1.Problem Statement

The current apps identified to support student-faculty interaction at universities, while they exist, are all limited and regimented in form and infrequently used by students as they search for assistance or information, most still use help desks or a website. Artificial intelligence (AI) implementations such as chat boots, voice-active agents, multilingual natural language processing (NLP), and predictive scheduling, previously when AI could not be a part of the solutions because students would defer access to faculty information or scheduling, will be more of a universal challenge for students to face with language barriers or disabilities. Today we can explore this issue as a research question with enhanced AI like advanced speech modeling (e.g. Whisper), transformer-based question answering; and more on predictive scheduling with the idea of transforming this university specific problem into a research question i.e. where are the AI agents have been tested in a research context; and devising a strategy of a manner in which AI could be systematically implemented at scale and inclusively with the higher education space.

2.Literature Review and Identified Gaps

Previous research and programs have demonstrated aspects of our vision, but there are significant gaps that this project intends to fill:

- **Restricted Usefulness of Campus Apps:** Many universities' mobile apps have limited capabilities and engagement with students [1]. For example, a 2021 study showed that "the majority of current apps used across Universities ... are limited in their capabilities and not very popular among students" [1]. **Gap:** Several apps remain unintegrated (faculty directory, campus maps, notifications). **Solution:** Our solution integrates important services (faculty directory, announcements, campus map) into one app to be useful and used.
- **Voice Interaction Capability Exists and Is Underutilized:** Students often use GUIs or text-only chat interfaces on school portals, and voice interfaces have rarely been applied. However, research on voice assistants indicates positive aspects: voice assistants allow users to multitask, provide a more natural interaction, typically have higher accuracy features than text-based interfaces [4] [5]. For example, in their research, Ellucian states: "Voice interaction is something that is especially interesting and engaging, ... a conversation will always, by its nature, provide a richer experience ... a relatively greater depth of emotional satisfaction" [4]. **Gap:** Voice command and speech recognition functions have not been built into university systems. **Solution:** Integrate ASR (speech-to-text) and TTS (text-to-speech) so that it is hands-free. We will use a leading ASR (e.g., OpenAI's Whisper, which was trained on 680,000+ hours of multilingual data [6]) to accomplish this.
- **Monolingual Interfaces:** Many campuses are linguistically diverse, but student support apps are often only in one language. Ralston et al. developed a multilingual voice chatbot by integrating IBM Watson into a student support system [7]. **Gap:** Existing solutions normally do not offer multilingual or multi-dialect services. **Solution:** Our chatbot design

includes multilingual support with NLP. An example of this would be Rasa's customizable pipelines with support for several languages [8]. The system will offer English and one or more local languages (using cloud translation APIs) when needed.

- **Scaling Peer Mentor Usage:** A literature review found universities are slow to transition to virtual peer-to-peer interactions despite benefits (24/7 access, familiarity) [6] [2]. In fact, “universities... have not yet recognized the value of implementing a peer support initiative” [2]. **Gap:** No institutions have provided scalable, non-faculty peer-to-peer support (for campus services, safety, admin questions). **Solution:** We'll build a peer mentor space in the app where an intelligent virtual peer will respond to common campus-related questions. The mentor space will allow for both text and video so the virtual peer can assist students anywhere.
- **Siloed “Smart” Systems:** Some projects (e.g., a smart mirror information kiosk [9]) show the potential of voice assistant technology on campus, but they remain location-based devices. **Gap:** one cannot use the information kiosk off-campus. **Solution:** Ensure students have the same smart-assistant capabilities anywhere (on- or off-campus) via mobile or web-based systems.

Each of these gaps—feature-limited apps [1], no voice/multilingual functionality [4] [8], and underutilized chatbots [2]—will be addressed by the proposed system, integrating them into a single platform leveraging modern AI.

3.Aims and Objectives

This project aims not only to design and develop a smart, scalable, web- and mobile-responsive app for faculty access and student support, but also to study and evaluate how AI capabilities can enhance inclusiveness, efficiency, and personalization in higher education. It will systematically integrate AI-enhanced features—multilingual voice interaction, AI-based timetable parsing, retrieval-augmented Q&A, and predictive scheduling—into one consolidated platform. Specifically, the project will benchmark a variety of AI models (e.g., Whisper vs. Google Speech for voice recognition; GPT-4 vs. BERT for Q&A; reinforcement learning vs. optimization for scheduling); assess their performance on campus; and iteratively identify the best combinations of tools to improve faculty-student communication, campus navigation, and management of academic/non-academic tasks.

Objectives:

1. **User Interface and Accessibility:** Developing a responsive user interface (UI) for students, lecturers, and administrators on web and mobile. Implement role-based access (RBAC) for different user types (students, staff, admins). Making accessibility compliant (WCAG compliant, multilingual etc.)
2. **Intelligent Timetable and Course Content:** Allow Faculty to upload class/appointment timetables. Allow students to auto-generate individual timetables. Communicate anytime there is a change to a class, or class is cancelled, and provide live notifications.

3. **Facility and Resource Usage:** Create a live availability mapping system for lecture halls, labs, library, etc. Implement a Resource Booking process/ system (rooms, equipment). Create orientation maps that can be interactive.
4. **Staff and Lecturer Directory:** Create searchable directory with office location, and availability of staff, and ways to make contact. Created an appointment booking system, linked to staff calendars and availability.
5. **AI Chatbot and Voice Interaction:** Voice input (speech-to-text) and audio output (text-to-speech) should provide hands-free interaction usefulness. Include links to FAQs and contextual help as references for faculty use in connection with the platform.
6. **Notifications and Alerts:** Push real-time alerts for events, emergencies, and academic deadlines. Enable customizable reminders (assignments, exams, appointments). Notify students of faculty activities or schedule changes.

4. Methodology

The project will follow a user-centered iterative development approach, where research experimentation and evaluation is key. Phase 1 (Months 1-2): Requirements analysis, stakeholder interviews, and an initial system design for interface and functionality. Create wireframes and user interface prototypes for a mobile/web interface, then implement the wireframed interfaces with functionality for each interface in Agile Sprints. Each phase of the project includes research tasks (benchmark metrics of ASR models on accuracy, comparing methods of timetable and calendar parsing, assessing the NLP quality of chatbot applications, etc.).

The key implementation phases include back-end application programming interface, front-end user interface, chatbot and voice interface applications, and predictive scheduling. Each of the above listed features will include user studies and experiments with a sampling of faculty and student users to collect usability feedback and assess model outputs. Outcomes from studies and experiments will be documented, including research notes, experiment results, results of performance comparisons, and usability of findings.

Standard software engineering aspects (version control, peer-review, automated testing) will provide quality to the technological system, while the research aspects and methodology will provide scientific rigor around the measurement and evaluation, and contribute to replicability. By the End of Month 12 the system will be fully test, ready for cloud deployment, and piloted to collect empirical data for publication, as well as practical validation.

1. System Implementation

Here is a list of recommended technologies for a solution that is scalable and maintainable:

- **Front end - Web and Mobile:** React (for web) and React Native (for mobile) - both share JavaScript as the language of development, both have large communities' help, and both help their user to quickly prototype UIs (user interfaces) similarly. React and React Native are the stereotypes for start-ups.

- **Backend:** Node.js with Express (or sometimes Factly)- event driven, non-blocking JavaScript runtime. A new technology that supports high concurrent load [10]. A JSON friendly Node.js runtime plus React and Express offers a more unified development team with low development overhead.
- **Database:** A new cloud database. PostgreSQL (supports foreign key) for relational data. Mongo DB for cases where the schema should be more lenient. Both solutions are accepted everywhere in industry; for example, PostgreSQL standards have excellent ACID compliance, cloud scalability (Amazon RDS), and there is also a cloud- hosted PostgreSQL which is widely available now (Atlas).
- **Cloud Infrastructure:** New solutions generally mean hosted at AWS (or Azure/GCP), but you can take advantage of in-house services (EC2/containers, S3 storage, managed databases), all of which can be scaled! Make sure to build a continuous integration pipeline and total deployment pipeline for software builds and deployments (like GitHub Actions + AWS Amplify etc.).

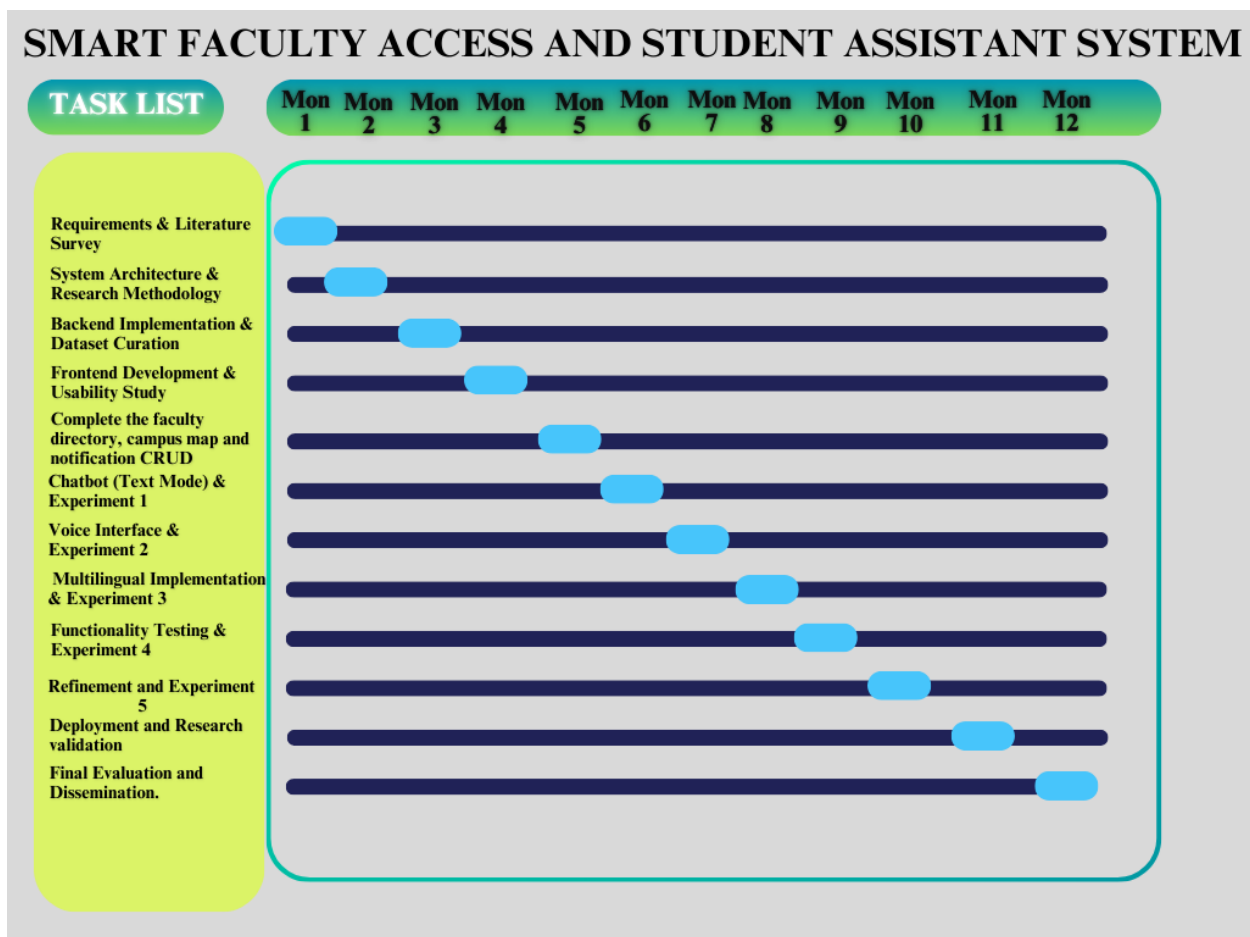
2.AI and NLP Tools

This project entails the integration of several different AI components that we have assessed with global alternatives to find the best suited match for application in the higher education context:

- **Chatbot Framework (Rasa):** Chatbot Framework (Rasa): We selected Rasa because it is open-source, readily customizable, and supports multilingual NLU pipelines. Furthermore, we have found through prior work efforts (see refs), that most university chatbots are rule-based and narrow. Rasa enables custom tokenizers, API integration, and retraining using university-specific data (i.e., FAQs, schedules, policies) which is not possible with closed-source platforms we evaluated. We will test Rasa pipelines against other framework options (Dialogflow, IBM Watson Assistant) along the lines of multilingual options, accuracy, and most importantly, usefulness to students.
- **Language Models (GPT-4 and Transformers):** Language Models (GPT-4 and Transformers): "Lead Little Models" (LLMs) such as GPT-4 track better on more complicated Question and Answer (Q&A), but generate narrative (ex. course details, policy orientations) richer in context [2]. However, there are related concerns about bias, accuracy, and privacy [3]. We will benchmark comparing GPT-4 to open-source transformer models (Bert, Robert, etc.). and benchmark retrieval-augmented Q&A. The research question we are posing is whether GPT-4 would significantly outpace lightweight, on-device models for academic Q&A and associated trade-offs with quality, latency and privacy.
- **Speech-to-Text (Whisper vs Cloud APIs):** Whisper (OpenAI) is a multilingual ASR trained on 680,000 hours of audio and robust to accent and noise [6]. Whisper can run locally (no vendor lock-in) unlike proprietary APIs (Google, Azure, AWS). We will also compare Whisper to commercial ASR APIs on accuracy, latency, and multilingual performance in noisy campus scenarios. This study is warranted because there is limited literature systematically addressing ASR in higher education.

- **Text-to-Speech (TTS):** Current services have realistic TTS, (Google Cloud TTS, Amazon Polly, etc.) so have open-source models. Quality TTS can improve accessibility for learners with sight issues. We will compare TTS services by voice clarity, multilingual capacity, user experience, and we will assess whether our open-source TTS could perform as well as cloud-managed services in academic contexts.
- **Multilingual NLP and Translation:** Campuses often have language diversity throughout. We will employ cloud APIs for translation services (Google, AWS) and open-source multilingual models (XLM-R, mBERT). We will compare performance trade-offs (accuracy, latency, offline) for cloud managed translation versus on-device multilingual models, to determine which better affords inclusive in academy settings.
- **Integration and Security:** Use OAuth2/JWT and AWS Cognito for authentication/security. We will compare the data privacy trade-offs from our research perspective of cloud LLMs/translation versus on-device models. Finally, we will benchmark and document the integration of conversational AI, multilingual NLP, and predictive models to serve practical and academic purposes.

5.12-Month Timeline and Milestones



- **Month 1: Requirements & Literature Survey.** Identify requirements through stakeholder interviews and a literature survey of AI in education. Identify any research gaps that exist. Write user stories and confirm project scope. Set up development environment, version control, research notebook, etc. (We will also develop a tangible deliverable system by discussing contents.).
- **Month 2: System Architecture & Research Methodology.** Design system architecture and UI/UX prototypes of the educational chatbot (both web and mobile). Confirm the tech stack and design comparative experiments (e.g., Whisper vs. Google Speech, GPT-4 vs. BERT). Define evaluation metrics (accuracy, latency, usability) and overall research methodology.
- **Month 3: Backend Implementation & Dataset Curation.** Start backend code development (Node.js/Express, PostgreSQL/Gravel, etc.). Provide coding for user/role authentication, database model, etc. Curate research dataset including faculty schedules, FAQs, policies, etc. to train/evaluate the chatbot and timetable parser components.
- **Month 4: Frontend Development & Usability Study.** Frontend development & Usability study. Initiate mobile/web frontend development using React/React Native. Conduct preliminary usability study on navigation and accessibility (e.g., using prototypes). Document baseline metrics for accessing navigation and accessibility for future studies.
- **Month 5: Complete the faculty directory, campus map and notification CRUD.** Start baseline tests of timetable parsing methods comparing rule versus OCR+NLP to capture performance benchmarks.
- **Month 6: Chatbot (Text Mode) & Experiment 1.** Build the chatbot in text mode using Rasa. Document intents/entities and graft in real schedules (office hours, new hours). Execute Experiment 1: Base Rasa NLU versus custom NLU, reporting accuracy across languages.
- **Month 7: Voice Interface & Experiment 2.** Integrate speech-to-text (Whisper, Google Speech API) and text-to-speech. Execute Experiment 2: ASR benchmarking of campus-like noisy settings with multi-lingual accents. Report accuracy, latency, and student satisfaction.
- **Month 8: Multilingual Implementation & Experiment 3.** Implement multi-lingual NLP using cloud APIs (Google Translate, AWS), and transformers (XLM-R, mBERT). Execute Experiment 3: Compare translation/NLP with cloud APIs versus on device. Report on accuracy, latency, and inclusiveness.
- **Month 9: Functionality Testing & Experiment 4.** Complete integration and unit testing of UI, backend APIs, chatbot and voice capabilities. Conduct a controlled user study with students, obtain quantitative (accuracy, latency, BLEU scores) and qualitative (UX surveys).

- **Month 10: Refinement and Experiment 5.** Refine chatbot knowledge and pipelines for voice/multilingual text to speech by user input (nits/feedback). Experiment 5: Q&A systems (GPT-4 vs. BERT vs. Retrieval-Augmented Generation) to evaluate: accuracy, trust, latency.
- **Month 11: Deployment and Research validation.** Deploy the system (cloud - AWS/Azure). Validation study comparing lab offer and real-world student deployment. User manuals, technical documentation, research articles.
- **Month 12: Final Evaluation and Dissemination.** Final evaluation and dissemination. Pilot with students and faculty (surveys; tracking use; value added). Aggregate data for research validation. Final report; research documents; handover project. Document for conference/journal submission on AI and education.

6.Ethics and Privacy

Since the system will be able to keep personal data and use voice data, ethics and privacy breach will need to be considered. To establish trust with the user and uphold principles for responsible AI, we will implement a range of policies and safeguards:

Data Management Procedures:

- **Where data will be stored:** All user data (login information, student schedules, and usage logs) will be stored in a secure, encrypted database provided by a reliable cloud service (e.g., Firebase/Atlas) and in compliance with relevant data protection regulations.
- **Data retention period:** The exact data will be retained only for as long as necessary (e.g., a single academic year, or project cycle) and deleted/anonymized immediately when the necessary period has concluded.
- **Access control:** Data can only be accessed by authorized users based on their position (e.g., student, lecturer, administrator) with access to relevant parts of the data. Developers and system administrators will have limited access for debugging or maintenance purposes based on role-based authentication.
- **How voice input is handled:** Voice commands will always be at player's control (e.g., will be indicator lights for "recording," will always be an explicit step to opt in), and raw audio saved to baseline will be held perpetuity unless it is needed to improve the system (with informed consent). Users can mute microphones and/or disable microphones, so that "earshot" recording is not accidentally recorded.

Bias and Fairness Measurements:

We will measure the chatbot to ensure it does not produce biased or harmful content. Measurement Metrics: We will measure performance and fairness using language-specific metrics, including:

- * The accuracy and error rate of ASR/NLP for Sinhala, Tamil, and English input.
- * Relevance and appropriateness of responses as rated by raters from different demographic and linguistic identities.

* Fairness indicators, including measuring gaps in error rates between languages (e.g., word error rate difference of < X%).

Exploratory evaluation will provide evidence as to whether multilingual NLP and TTS tools remove barriers for non-native speakers and students with disabilities.

Privacy-Preserving Approaches: We will compare the different implementations (ASR/NLP on-device or via cloud service) and evaluate trade-offs with respect to accuracy, latency, privacy, and trustworthiness. This exploration aims to contribute not only to a functional system, but also to initial research on how to ethically adopt responsible AI in higher education.

7. References

- [1] S. Z. Sweidan, S. S. Abu Laban, N. A. Alnaimat, and K. A. Darabkh, "SIAAA-C: A student interactive assistant android application with chatbot during COVID-19 pandemic," *Computer Applications in Engineering Education*, vol. 29, no. 6, Apr. 2021, doi: <https://doi.org/10.1002/cae.22419>.
- [2] K. Peyton, S. Unnikrishnan, and B. Mulligan, "A review of university chatbots for student support: FAQs and beyond," *Discover Education*, vol. 4, no. 1, Jan. 2025, doi: <https://doi.org/10.1007/s44217-025-00397-7>.
- [3] William Seymour, Xiao Zhan, Mark Coté, and Jose Such, "A Systematic Review of Ethical Concerns with Voice Assistants," *Montreal AI Ethics Institute*, Sep. 03, 2023. <https://montrealethics.ai/a-systematic-review-of-ethical-concerns-with-voice-assistants/>
- [4] "Coming to a campus near you: Voice assistants | Ellucian," *Ellucian*, 2025. <https://www.ellucian.com/blog/coming-campus-near-you-voice-assistants> (accessed Aug. 20, 2025).
- [5] ". Natural Sciences Publishing.," *www.naturalspublishing.com*. <https://www.Publishing:shing.com>
- [6] OpenAI, "Introducing Whisper," *Openai.com*, 2022. <https://openai.com/index/whisper/>
- [7] K. Ralston, Y. Chen, H. Isah, and F. Zulkernine, "A Voice Interactive Multilingual Student Support System using IBM Watson," *arXiv.org*, 2019. <https://arxiv.org/abs/2001.00471>
- [8] "Rasa vs Dialogflow vs Lex - Chatbot Features Comparison," *Ideas2it.com*, 2024. <https://www.ideas2it.com/blogs/battle-of-the-bots-rasa-vs-google-dialogflow-vs-aws-lex>
- [9] "International Journal of Electrical and Computer Engineering (IJECE)," *ijece.iaescore.com*.

<https://ijece.iaescore.com/index.php/IJECE>

[10] N. Khandelwal, "Top 10 Benefits of Using Node.js for Web Development," *VLink*, Aug. 05, 2024. <https://vlinkinfo.com/blog/benefits-of-using-nodejs-for-web-development> (accessed Aug. 20, 2025).

[11] Tamusa.edu, 2025. <https://www.tamusa.edu/academics/ai-resources/documents> (accessed Aug. 20, 2025).

[12] "ChatGPT Education," *Openai.com*, 2015. <https://openai.com/chatgpt/education/>